

< 연습문제 #02 > (기본신경망 - 분류)

1. 클래스 가중치를 다음과 같이 수정하는 코드를 완성하고, 그 결과를 정리하시오.

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
import keras

FC = pd.read_csv('FC_new.csv')
feature = FC[['v', 'a', 'sin']].to_numpy()
target = FC['FC_Label'].to_numpy()
train_input, test_input, train_target, test_target = train_test_split(feature, target, test_size=0.4, random_state=42)
val_input, test_input, val_target, test_target = train_test_split(test_input, test_target, test_size=0.5, random_state=42)

model = keras.Sequential()
model.add(keras.layers.Dense(100, activation='relu'))
model.add(keras.layers.Dropout(0.3))
model.add(keras.layers.Dense(100, activation='relu'))
model.add(keras.layers.Dropout(0.3))
model.add(keras.layers.Dense(100, activation='relu'))
model.add(keras.layers.Dropout(0.3))
model.add(keras.layers.Dense(7, activation='softmax'))

class_counts = np.bincount(train_target.astype(int))
total_samples = len(train_target)
num_classes = len(class_counts)
class_weights = {}
for i, count in enumerate(class_counts):
    class_weights[i] = round((total_samples / (num_classes * count)), 3)

print(class_weights)
class_weights[6] = 100.0
print(class_weights)

model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
history = model.fit(train_input, train_target, epochs=10, validation_data=(val_input, val_target), class_weight=class_weights)
model.summary()
model.save('FC_class_weighted2.keras')

plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.xlabel('epoch')
plt.ylabel('loss')
plt.legend(['train', 'val'])
plt.title('Loss with Class Weights')
plt.show()
```

(결과 정리)

변경 전 : class_weights = {0: np.float64(1.77), 1: np.float64(0.359), 2: np.float64(0.602), 3: np.float64(0.998),
4: np.float64(1.633), 5: np.float64(2.75), 6: np.float64(158.502)}

변경 후 : class_weights = {0: np.float64(1.77), 1: np.float64(0.359), 2: np.float64(0.602), 3: np.float64(0.998),
4: np.float64(1.633), 5: np.float64(2.75), 6: 100.0}

```

from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score
import matplotlib.pyplot as plt
import seaborn as sns

model.evaluate(test_input, test_target)
test_pred_weighted = np.argmax(model.predict(test_input), axis=1)
cm_weighted = confusion_matrix(test_target, test_pred_weighted)
accuracy_weighted = round(accuracy_score(test_target, test_pred_weighted), 3)
precision_weighted = precision_score(test_target, test_pred_weighted, average=None, zero_division=0).round(3)
recall_weighted = recall_score(test_target, test_pred_weighted, average=None, zero_division=0).round(3)
f1_weighted = f1_score(test_target, test_pred_weighted, average=None, zero_division=0).round(3)

print("Confusion Matrix:\n", cm_weighted)
print(f"Accuracy: {accuracy_weighted}")
print(f"Precision: {precision_weighted}")
print(f"Recall (Sensitivity): {recall_weighted}")
print(f"F1 Score: {f1_weighted}")
plt.figure(figsize=(8, 6))
sns.heatmap(cm_weighted, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Confusion Matrix Heatmap (Weighted Model)")
plt.show()

```

(결과 정리) class_weight for class 6 = 100.0

(학생마다 조금씩 다르게 나옴)

model.evaluate(test_input, test_target) = - accuracy: 0.7014 - loss: 0.7335

Confusion Matrix:

```

[[1044   1  47   2   4   0   0]
 [ 860 4361 552  22   8   0   0]
 [ 459  97 2310 446  77   2   0]
 [  34   1  469 1098 450  38   2]
 [   7   0   36  230 816 220   0]
 [   3   0   3   14 130 560   8]
 [   0   0   0   0   2  11   0]]

```

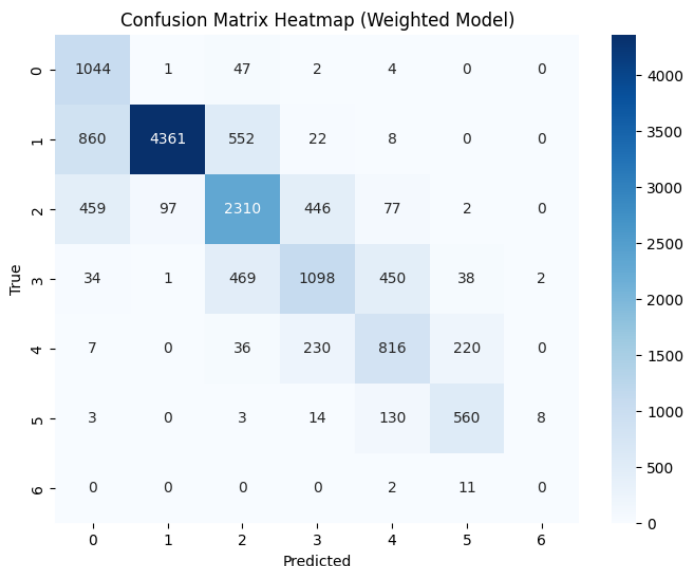
Accuracy: 0.706

Precision: [0.434 0.978 0.676 0.606 0.549 0.674 0.]

Recall (Sensitivity): [0.951 0.752 0.681 0.525 0.623 0.78 0.]

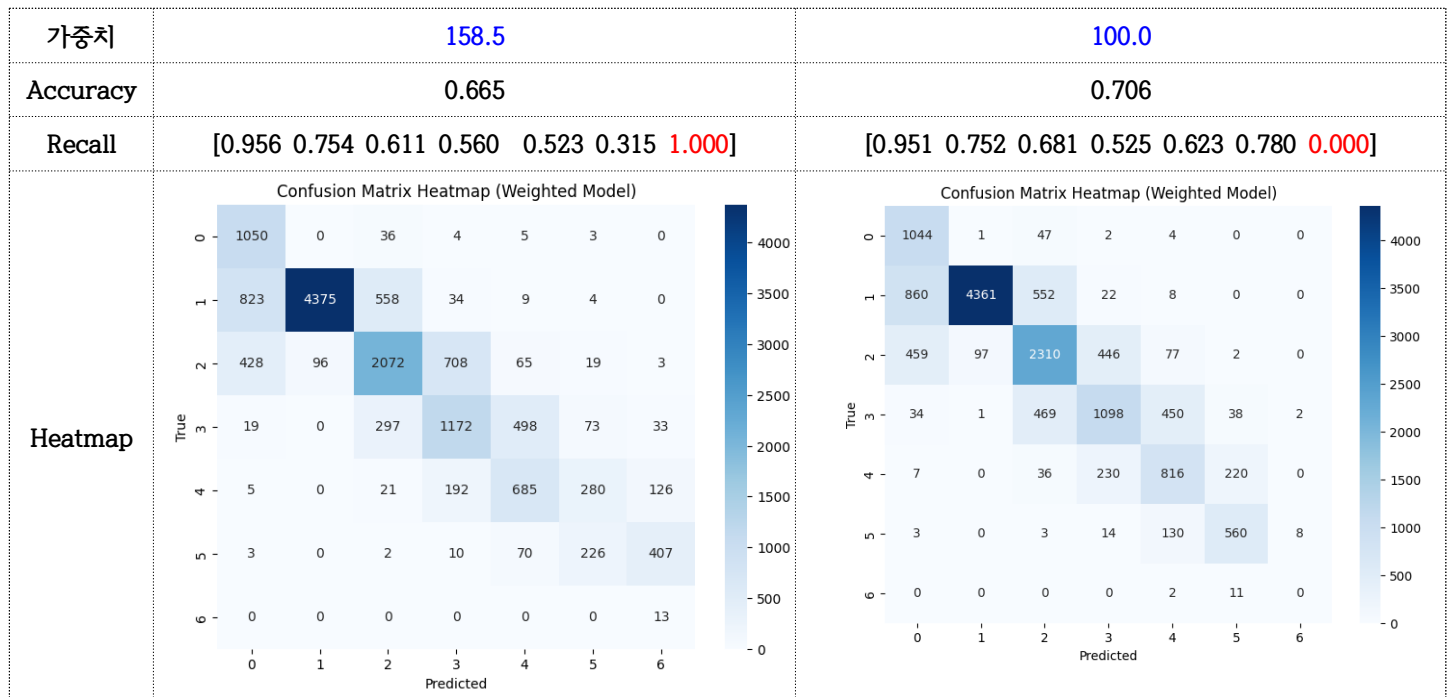
F1 Score: [0.596 0.85 0.679 0.562 0.584 0.723 0.]

Heatmap



2. class 6에 대한 가중치가 다음과 같을 때 그 성능을 비교, 정리하시오.

(학생마다 조금씩 다르게 나옴)



3. 클래스 가중치를 다음과 같이 구하는 코드에 대해 답하시오.

```
import numpy as np
```

```
class_counts = np.array([32, 23, 30, 15])
total_samples = class_counts.sum()
num_classes = len(class_counts)
print(class_counts, total_samples, num_classes)
```

```
class_weights1 = {}
class_weights2 = {}
```

```
for i, count in enumerate(class_counts):
    class_weights1[i] = round((total_samples / (num_classes * count)), 3)
    class_weights2[i] = round((total_samples / (count)), 3)
```

```
print(class_weights1)
print(class_weights2)
```

```
[32 23 30 15] 100 4
{0: np.float64(0.781), 1: np.float64(1.087), 2: np.float64(0.833), 3: np.float64(1.667)}
{0: np.float64(3.125), 1: np.float64(4.348), 2: np.float64(3.333), 3: np.float64(6.667)}
```

(1) 데이터가 가장 적은 클래스는? **클래스 3**

(2) 전체 데이터 개수는? **100**

(3) 클래스의 종류는 몇 개인가? **4**

(4) 클래스 가중치1 = {0: np.float64(0.781), 1: np.float64(1.087), 2: np.float64(0.833), 3: np.float64(1.667)}

(5) 클래스 가중치2 = {0: np.float64(3.125), 1: np.float64(4.348), 2: np.float64(3.333), 3: np.float64(6.667)}

(6) 가중치2의 단점은 무엇인가? **모든 클래스의 가중치가 1보다 크다.**

4. 클래스가 3개인 분류 모델의 혼동행렬이 다음과 같을 때, 정확도, 정밀도, 재현도, f1 점수를 각각 구하시오.

Confusion Matrix		Predicted Label			sum	재현도 (→) (recall, sensitivity)
		0	1	2		
Actual Label	0	60	15	10	85	60 / 85 = 0.706
	1	25	75	10	110	75 / 110 = 0.682
	2	15	20	65	100	65 / 100 = 0.650
sum		100	110	85	295	
정밀도 (↓) (precision)		60 / 100 = 0.600	0.682	0.765	정확도(accuracy) = (60 + 75 + 65) / 295 = 0.678	
R		0.706	0.682	0.650		
f1-score = 2*P*R / (P+R)		0.649	0.682	0.703		

Accuracy = 0.678 : 전체적으로 예측(분류) 정확도 67.8% 수준
Precision = [0.600, 0.682, 0.765] : 클래스 2로 예측하는 경우 맞을 가능성 높음 (76.5%)
Recall = [0.706, 0.682, 0.650] : 클래스 0에 대한 예측 정확도가 높음 (70.6%)
f1_score = [0.649, 0.682, 0.703] : 클래스 2에 대한 분류 정확도 높음 (70.3%)

5. 다음 지표 혹은 용어에 대해 설명하시오.

- (1) 혼동행렬(confusion matrix) : 클래스별로 예측 데이터 개수
- (2) 정확도(accuracy) : 전체 데이터 기준으로 정확하게 분류한 비율
- (3) 정밀도(precision) : 예측 클래스 기준으로 정확하게 분류한 비율
- (4) 재현도(민감도)(recall, sensitivity) : 실제 클래스 기준으로 정확하게 분류한 비율
- (5) f1-score : 정밀도와 재현도의 조화평균
- (6) 클래스 가중치 : 클래스별 데이터에 반비례하는 가중치를 클래스별 손실값에 곱해서 계산
전체 데이터 개수 / (클래스 개수 * 클래스 데이터 개수)